

Simulation der Navier-Stokes-Gleichungen mit Hilfe von künstlicher Intelligenz

Seminararbeit

Autoren: Anna Sonnleitner, Emanuel Steininger, Martin Strobl
Studium: Technische Mathematik
Betreuer: Univ.Prof. Dipl.-Ing. Dr.techn. Joachim Schöberl
Institut: Institut für Numerische Mathematik
Datum: April 24, 2026

Contents

1	Einleitung	3
2	Die Navier-Stokes Gleichungen	4
2.1	Eine kurze Herleitung	4
2.1.1	Grundannahmen	4
2.1.2	Massenerhaltung	4
2.1.3	Impulserhaltung	4
2.1.4	Navier-Stokes-Gleichungen	4
2.2	schwache Formulierung	5
2.2.1	Funktionenräume	5
2.2.2	Testfunktionen	5
2.2.3	schwache Form	5
2.3	Randbedingungen	6
2.4	Reynoldszahlen	6
3	Finite-Elemente-Methode (FEM)	8
3.1	Was ist FEM?	8
3.2	Vorgehensweise in der FEM	9
3.3	Illustration an einfachem Beispiel:	10
3.4	Randbedingungen in der FEM	12
4	Simulation mit FEniCS	14
4.1	Softwareüberblick	14
4.2	Setup und Codeaufbau	14
4.2.1	Aufbau	14
4.2.2	Funktionsweise	15
4.3	Fazit	16
5	Simulation mit NGSolve	17
5.1	Softwareaufbau	17
5.2	Simulation mit künstlicher Intelligenz	17
5.3	Fazit	18
6	Finite-Volumen-Methode (FVM)	19
6.1	Was ist die FVM?	19
6.2	Herleitung anhand eines einfachen Beispiels	19
6.3	Diskretisierungen im speziellen Fall	20
6.3.1	Der Quellterm	20
6.3.2	Der lokale Zeitterm	20
6.3.3	Der Konvektionsterm	21
6.3.4	Der Diffusionsterm	22
6.3.5	Unterschiedliche Interpolationsschemata	22
6.4	Gute Mesh-Konstruktion	24
6.5	Verfahren der Solver	24

6.5.1	Druckkorrektur	25
7	Simulation mit OpenFOAM	26
7.1	Überblick und Codeaufbau	26
7.1.1	Aufbau	26
7.1.2	Funktionsweise	27
7.2	Fazit	27
	Referenzen	28
	Eidesstattliche Erklärung	29

1 Einleitung

Was haben der Fluss einer Strömung durch einen Kanal, die Luftverdrängung eines Flugzeuges, und das Verhalten von Wasser in Stromschnellen gemeinsam? Sie können alle durch die Navier-Stokes-Gleichungen simuliert werden. Dieses komplexe, nichtlineare System von partiellen Differentialgleichungen wird in der modernen Mathematik viel diskutiert. Wegen der Nichtlinearität der Gleichungen und den, in der Realität oft schwierig zu handhabenden Randbedingungen, muss zum Lösen dieser Gleichungen oft auf rein numerische Verfahren zurückgegriffen werden.

Ziel dieser Arbeit ist die Simulation eines standardisierten Problems. In einem Kanal (0.41m x 2m) soll sich ein zylinderförmiges Hindernis (0.05m Radius) befinden, das von der Strömung berücksichtigt werden soll. Die Implementierung und Modellierung soll dabei möglichst ausschließlich von 'Künstlicher Intelligenz' (KI) oder im Englischen 'Artificial Intelligence' (AI) übernommen werden. Umgesetzt wurde das Projekt mit den drei Open-Source Frameworks FEniCS, NGSolve und OpenFOAM. Sowohl FEniCS als auch NGSolve stützen sich auf die Finite-Elemente-Methode, OpenFOAM hingegen verwendet die Finite-Volumen-Methode.

Der Fokus wird dabei auf den Vergleich von verschiedenen AI-Modellen bei den unterschiedlichen Frameworks gelegt. Stärken und Schwächen werden diskutiert und es wird analysiert, ob in Zukunft die Simulation rein mit AI möglich sein könnte.

2 Die Navier-Stokes Gleichungen

2.1 Eine kurze Herleitung

Im Folgenden wird eine kurze Herleitung der Navier-Stokes-Gleichungen für ein inkompressibles, viskoses Fluid angegeben, wobei auf die explizite Einführung des Spannungstensors verzichtet wird. Diese Darstellung ist kompakter und reicht für eine einführende Seminararbeit aus.

2.1.1 Grundannahmen

Es wird ein Newtonsches Fluid mit konstanter Dichte ρ und konstanter dynamischer Viskosität μ betrachtet. Das Fluid wird als Kontinuum modelliert und durch ein Geschwindigkeitsfeld $\mathbf{u}(\mathbf{x}, t)$ sowie ein Druckfeld $p(\mathbf{x}, t)$ beschrieben. Zusätzlich wirken äußere Volumenkräfte $\mathbf{f}$, beispielsweise durch Gravitation.¹

2.1.2 Massenerhaltung

Die Erhaltung der Masse wird durch die Kontinuitätsgleichung beschrieben. Für ein inkompressibles Fluid vereinfacht sich diese zu

$$\nabla \cdot \mathbf{u} = 0. \quad (1)$$

2.1.3 Impulserhaltung

Die Impulserhaltung folgt aus dem zweiten Newtonschen Gesetz. Die zeitliche Änderung des Impulses eines Fluidelements ist gleich der Summe der darauf wirkenden Kräfte. In lokaler Form ergibt sich

$$\rho \frac{D\mathbf{u}}{Dt} = -\nabla p + \mu \Delta \mathbf{u} + \rho \mathbf{f}, \quad (2)$$

wobei $\frac{D}{Dt}$ die materielle Ableitung bezeichnet. Diese setzt sich aus einem lokalen und einem konvektiven Anteil zusammen:

$$\frac{D\mathbf{u}}{Dt} = \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u}. \quad (3)$$

Der Druckterm $-\nabla p$ wirkt als Zwangskraft, die die Inkompressibilitätsbedingung erfüllt, während der viskose Term $\mu \Delta \mathbf{u}$ die innere Reibung des Fluids modelliert.

2.1.4 Navier-Stokes-Gleichungen

Durch Einsetzen der materiellen Ableitung in die Impulserhaltung erhält man das Gleichungssystem der inkompressiblen Navier-Stokes-Gleichungen:²

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} \right) = -\nabla p + \mu \Delta \mathbf{u} + \rho \mathbf{f}, \quad (4)$$
$$\nabla \cdot \mathbf{u} = 0.$$

¹[Jün20]
²[RVe19]

Dieses System bildet die Grundlage für die numerischen Simulationen, die in den folgenden Kapiteln mittels verschiedener Diskretisierungsmethoden untersucht werden.

2.2 schwache Formulierung

Zur numerischen Lösung der Navier-Stokes-Gleichungen mittels der Finite-Elemente-Methode ist es notwendig, die starke Form der Gleichungen in eine schwache (variationale) Formulierung zu überführen. Diese erlaubt es, Lösungen in geeigneten Funktionenräumen zu suchen, in denen klassische Ableitungen nicht überall existieren müssen.

2.2.1 Funktionenräume

Sei $\Omega \subset \mathbb{R}^d$ ein beschränktes Gebiet mit Rand $\partial\Omega$. Die Geschwindigkeit $\mathbf{u}$ wird in einem geeigneten Vektorfunktionenraum gesucht, typischerweise

$$\mathbf{u} \in V := \{ \mathbf{v} \in H^1(\Omega)^d \mid \mathbf{v} = \mathbf{0} \text{ auf } \Gamma_D \}, \quad (5)$$

während der Druck in einem Skalarfunktionenraum

$$p \in Q := L^2(\Omega) \quad (6)$$

liegt. Dabei bezeichnet $\Gamma_D \subseteq \partial\Omega$ den Teil des Randes, auf dem Dirichlet-Randbedingungen (z.B. No-Slip) vorgegeben sind.

2.2.2 Testfunktionen

Ausgehend von der starken Form der inkompressiblen Navier-Stokes-Gleichungen werden die Gleichungen mit geeigneten Testfunktionen $\mathbf{v} \in V$ bzw. $q \in Q$ multipliziert und über das Gebiet Ω integriert. Für die Impulsgleichung ergibt sich

$$\int_{\Omega} \rho \left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} \right) \cdot \mathbf{v} dx = \int_{\Omega} (-\nabla p + \mu \Delta \mathbf{u} + \rho \mathbf{f}) \cdot \mathbf{v} dx. \quad (7)$$

Die Inkompressibilitätsbedingung wird analog mit einer skalaren Testfunktion q behandelt.

2.2.3 schwache Form

Durch partielle Integration mithilfe des 'Satz von Gauß', um den Ableitungsgrad der Funktionen zu reduzieren, erhält man folgende schwache Formulierung für die inkompressiblen Navier-Stokes-Gleichungen:

$$(\mathbf{u}_t, \mathbf{v}) + ((\mathbf{u} \cdot \nabla) \mathbf{u}, \mathbf{v}) + (\nabla \mathbf{u}, \nabla \mathbf{v}) - (p, \nabla \cdot \mathbf{v}) = 0 \quad (8)$$

$$(q, \nabla \cdot \mathbf{u}) = 0. \quad (9)$$

$\mathbf{u}$ und p sollen hierbei aus den oben gewählten Funktionenräumen stammen. Mithilfe dieser Formulierung lassen sich die Gleichungen auf ein Finite-Elemente-Konzept übertragen und somit auch numerisch stabil implementieren.

2.3 Randbedingungen

Für die eindeutige Lösbarkeit des Strömungsproblems müssen zusätzlich zu den Navier-Stokes-Gleichungen, geeignete Bedingungen auf dem Rand $\partial\Omega$ des Rechengebiets vorgegeben werden. Der Rand wird hierzu in drei disjunkte Teile zerlegt:

$$\partial\Omega = \Gamma_W \cup \Gamma_{\text{in}} \cup \Gamma_{\text{out}}, \quad (10)$$

wobei Γ_W stationäre Wände, Γ_{in} den Einlass und Γ_{out} den Auslass bezeichnen.

No-Slip-Randbedingung an den Wänden: An stationären Wänden wird die klassische No-Slip-Randbedingung verwendet, welche besagt, dass die Fluidgeschwindigkeit relativ zur Wand verschwindet:

$$\mathbf{u} = \mathbf{0} \quad \text{auf } \Gamma_W. \quad (11)$$

Diese Randbedingung wird in der schwachen Formulierung 'stark' vorgegeben, ist also bereits in der Definition des Geschwindigkeitsraums V berücksichtigt.

Einlassrandbedingung: Am Einlass wird eine konstante Geschwindigkeit vorgeschrieben:

$$\mathbf{u} = \mathbf{u}_{\text{in}} \quad \text{auf } \Gamma_{\text{in}}, \quad (12)$$

wobei $\mathbf{u}_{\text{in}}$ ein gegebener, zeitlich konstanter Geschwindigkeitsvektor ist. Auch diese Dirichlet-Randbedingung wird stark in den Funktionenraum integriert.

Auslassrandbedingung (Do-Nothing): Am Auslass wird eine sogenannte Do-Nothing-Randbedingung verwendet. Diese ergibt sich natürlich aus der schwachen Formulierung, indem auf Γ_{out} keine zusätzlichen Randterme vorgeschrieben werden. Vereinfacht entspricht dies der natürlichen Randbedingung

$$\nabla \mathbf{v} \cdot \mathbf{n} - p \mathbf{n} = \mathbf{0} \quad (13)$$

Die Kombination dieser Randbedingungen stellt ein physikalisch sinnvolles und numerisch gut handhabbares Setup für die in den folgenden Kapiteln betrachteten Simulationen dar.

2.4 Reynoldszahlen

Zur Charakterisierung von Strömungen wird häufig die Reynoldszahl verwendet, eine dimensionslose Kennzahl, die das Verhältnis von Trägheitskräften zu viskosen Kräften beschreibt. Sie ist definiert als

$$\text{Re} = \frac{\rho U L}{\mu} = \frac{U L}{\nu}, \quad (14)$$

wobei U eine charakteristische Geschwindigkeit, L eine charakteristische Längenskala und $\nu = \mu/\rho$ die kinematische Viskosität des Fluids ist.

Die Reynoldszahl bestimmt das Verhalten der Strömung. Kleine Reynoldszahlen entsprechen laminaren Strömungen, während bei großen Reynoldszahlen Trägheitseffekte überwiegen und turbulente Strömungen auftreten können.

Durch Einführung dimensionsloser Variablen

$$\tilde{\mathbf{x}} = \frac{\mathbf{x}}{L}, \quad \tilde{t} = \frac{U}{L}t, \quad \tilde{\mathbf{u}} = \frac{\mathbf{u}}{U}, \quad \tilde{p} = \frac{p}{\rho U^2}, \quad (15)$$

lassen sich die Navier-Stokes-Gleichungen in eine dimensionslose Form überführen. Nach Einsetzen und Weglassen der Tilden ergibt sich

$$\mathbf{u}_t + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \frac{1}{\text{Re}}\Delta\mathbf{u} \quad (16)$$

$$\nabla \cdot \mathbf{u} = 0. \quad (17)$$

In dieser Darstellung tritt die Reynoldszahl als einziger dimensionsloser Parameter auf und bestimmt maßgeblich das Verhalten der Lösung. Sie spielt daher sowohl in der theoretischen Analyse, als auch bei der numerischen Simulation von Strömungen eine zentrale Rolle.³

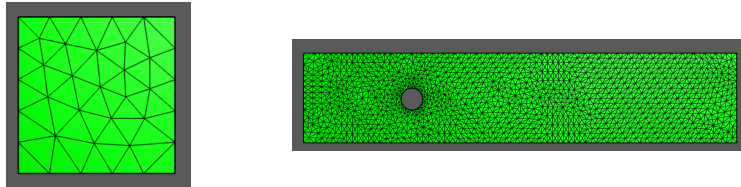
³[Alf24]

3 Finite-Elemente-Methode (FEM)

3.1 Was ist FEM?

*"It is the most powerfull and most popular numerical method for boundary value problems in more than one dimension."*⁴

Die Finite-Elemente-Methode ist ein numerisches Berechnungsverfahren zur Lösung von mathematischen, physikalischen, chemischen oder strukturellen Modellen. Sie wird im Maschinen- und Fahrzeugbau, in der Optik und Elektroindustrie, aber auch in der Chemie und Reaktionssicherheit angewendet. Der Begriff "finite Elemente" bezieht sich auf die Aufteilung des Berechnungsgebietes in endlich viele Teilkörper, mit möglichst einfacher Geometrie, oft Dreiecke oder Tetraeder, wodurch das ursprünglich analytische Problem zu einem algebraischen umgewandelt wird.⁵



Mithilfe von passenden Ansatzfunktionen wird dafür eine Lösung gesucht, die eine Annäherung an die eigentliche Lösung des analytischen Problems ist. Die Größe der endlich gewählten Teilkörper steht dabei direkt im Zusammenhang mit dem erhaltenen Rechenergebnis.⁶

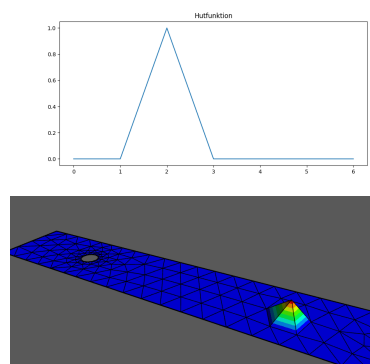


Abbildung 1: lineare Basisfunktionen

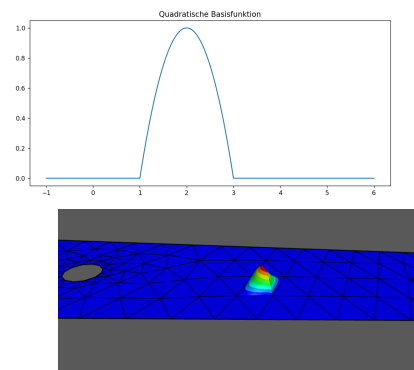


Abbildung 2: quadratische Basisfunktionen

⁴[Inn+25]: S. 105

⁵Alle in dieser Arbeit verwendeten Abbildungen wurden von den Verfassern selbst erstellt.

⁶[Sto]

3.2 Vorgehensweise in der FEM

Problemstellung: In der FEM möchte man eine partielle Differentialgleichung (PDE) mit gegebenen Randbedingungen lösen.

Die Finite-Elemente-Methode (FEM) arbeitet dabei mit der schwachen Formulierung der PDE, wofür schließlich eine Lösung u gesucht wird, die die schwache Form für alle v erfüllt, wobei u, v aus dem Sobolevraum $H^1(\Omega)$ sein sollen. Im Allgemeinen ergibt sich also folgende Problemstellung:

$$\text{suchen } u \in H^1(\Omega) : \quad A(u, v) = f(v) \quad \forall v \in H^1(\Omega)$$

Bei Problemen mit Dirichlet-Randbedingungen wird auch der Raum $H_0^1(\Omega)$ verwendet, was im Abschnitt 3.4 noch genauer bearbeitet wird.

Da für unendlich-dimensionale Räume im Allgemeinen keine Lösung berechnet werden kann, definiert man sich einen endlich-dimensionalen Teilraum

$$V_h \subset H^1(\Omega) \text{ mit } \dim V_h = N < \infty.$$

Dadurch erhalten wir die schwache Formulierung:

$$\text{suchen } u_h \in V_h : \quad A(u_h, v_h) = f(v) \quad \forall v_h \in V_h$$

Da in endlich-dimensionalen Räumen immer eine Basis $B = (b_1(x), b_2(x), \dots, b_N(x))$ existiert, also:

$$V_h = \text{span}[b_1, b_2, \dots, b_N]$$

können wir folgern das aufgrund der Basiseigenschaft u_h von folgender Form sein muss:

$$u_h = \sum_{i=1}^N u_i b_i(x)$$

wobei $u := (u_1, \dots, u_N) \in \mathbb{R}^N$. Aufgrund der Bilinearität bzw. Linearität der Abbildungen in der schwachen Formulierung reicht es nun nur noch die Basisfunktionen $b_1(x), \dots, b_N(x)$ als Testfunktionen zu betrachten. Dadurch erhalten wir als neue Problemstellung:

$$\text{suchen } u \in \mathbb{R}^N : \quad \sum_{i=1}^N A(b_i, b_j) u_i = f(b_j) \quad \forall j = 1, \dots, N$$

Und da wir zu Beginn passende und somit bekannte Basisfunktionen gewählt haben, können wir $A(b_i, b_j)$ und $f(b_j)$ für alle i, j berechnen, wodurch wir eine Matrix A und einen Vektor f erhalten:

$$A = (A(b_i, b_j))_{i,j=1}^N \text{ und } f = (f(b_j))_{j=1}^N$$

Dies führt uns zu einem linearen Gleichungssystem

$$\text{suchen } u \in \mathbb{R}^N : \quad A^T \mathbf{u} = f \quad \forall j = 1, \dots, N$$

Der Lösungsvektor $\mathbf{u}$ dieses Gleichungssystems besteht nun aus den Koeffizienten, für die Näherungslösung u_h .⁷

⁷[Schc], [Jün01], [Inn+25]: S.105-109

3.3 Illustration an einfachem Beispiel:

Wir wählen für die unten angegebene PDE auf einem 1D-Gebiet Ω mithilfe der FEM annähern.

Gegeben ist $\Omega = (0, 1) \subset \mathbb{R}$ mit der PDE $\Delta \mathbf{u} = \mathbf{f}$ wobei $\mathbf{f} \equiv -1$ und Randbedingungen $u(0) = u(1) = 0$

Zuerst berechnen wir die schwache Formulierung der PDE mithilfe des Satz von Gauß:

$$\mathbf{A}(\mathbf{u}, \mathbf{v}) = - \int_0^1 \nabla \mathbf{u} \nabla \mathbf{v} dx = \int_0^1 \mathbf{f} \mathbf{v} dx = \mathbf{f}(v)$$

Wir teilen nun unser Gebiet Ω in Teilgebiete auf mit $n = 5$ Knoten/Grenzpunkten $x_i = \frac{i}{n}$ für $i = 0, \dots, n$, also $\mathbf{x} = (0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1)$, womit wir äquidistante Stützstellen mit $h = \frac{1}{4}$ erhalten.

Nun bilden wir darauf unsere Basisfunktionen ϕ_i die linear sein müssen und für die folgende Bedingung gelten muss:

$$\phi_i(x_j) = \delta_{ij} \quad \forall i, j = 1, \dots, 5$$

wodurch wir folgende Funktionen erhalten:

$$\phi_1(x) = \begin{cases} \frac{x_2 - x}{h}, & x \in [x_1, x_2] \\ 0, & x \geq x_2 \end{cases}$$

für $i = 2, 3, 4$:

$$\phi_i(x) = \begin{cases} \frac{x - x_{j-1}}{h}, & x \in [x_{j-1}, x_j] \\ \frac{x_{j+1} - x}{h}, & x \in [x_j, x_{j+1}] \\ 0, & x \notin [x_{j-1}, x_{j+1}] \end{cases}$$

$$\phi_5(x) = \begin{cases} \frac{x - x_{j-1}}{h}, & x \in [x_4, x_5] \\ 0, & x \leq x_4 \end{cases}$$

unser Funktionenraum, in dem wir nun unsere Lösung suchen ist nun

$$V_h = \text{span}[x_1, \dots, x_5] \text{ mit gesuchter Lösung: } u_h(x) = \sum_{i=1}^5 \phi_i(x) u_i$$

wobei $u_i \in \mathbb{R}$ bzw $u = (u_1, u_2, \dots, u_5) \in \mathbb{R}^5$.

Da unsere Randbedingungen $u(0) = u(1) = 0$ gelten müssen, folgt $u_1 = 0 = u_5$ und unser Funktionenraum reduziert sich zu:

$$V_h = \text{span}[x_1, x_2, x_3] \text{ mit gesuchter Lösung: } u_h(x) = \sum_{i=2}^4 \phi_i(x) u_i.$$

Wie in Abschnitt 3.2 erklärt, erhalten wir aufgrund der Linearität bzw. Bilinearität unserer Abbildungen und der Tatsache, dass wir nur unsere Basisfunktionen als Testfunktionen betrachten müssen, folgendes:

$$\text{suchen } u \in \mathbb{R}^3 : \quad \sum_{i=2}^4 A(b_i, b_j) u_i = f(b_j) \quad \forall j = 2, 3, 4$$

Um das zu lösen, müssen wir $A(b_i, b_j)$ und $f(b_j)$ für alle i, j berechnen, wofür wir hier ein paar Beispiele vorrechnen.

Bemerkung: die ersten Ableitungen der Basisfunktionen sind folgende:

$$\phi'_i(x) = \begin{cases} \frac{1}{h}, & x \in [x_{i-1}, x_i], \\ -\frac{1}{h}, & x \in [x_i, x_{i+1}], \\ 0, & \text{sonst,} \end{cases} \quad i = 2, 3, 4,$$

$$\phi'_1(x) = \begin{cases} -\frac{1}{h}, & x \in [x_1, x_2], \\ 0, & \text{sonst,} \end{cases} \quad \phi'_5(x) = \begin{cases} \frac{1}{h}, & x \in [x_4, x_5], \\ 0, & \text{sonst.} \end{cases}$$

diese benötigen nun für die Berechnung von $A(b_i, b_j)$:

$$A_{22} = - \int_0^1 \phi'_2 \phi'_2 dx = - \left(\int_0^{\frac{1}{4}} \frac{1}{h^2} dx + \int_{\frac{1}{4}}^{\frac{1}{2}} \left(-\frac{1}{h}\right)^2 dx \right) = -\frac{1}{2h^2} = -8$$

$$A_{23} = - \int_0^1 \phi'_2 \phi'_3 dx = - \int_0^{\frac{1}{4}} \frac{1}{h} \cdot 0 dx - \int_{\frac{1}{4}}^{\frac{1}{2}} \left(-\frac{1}{h}\right) \cdot \frac{1}{h} dx - \int_{\frac{1}{2}}^{\frac{3}{4}} 0 \cdot \left(-\frac{1}{h}\right) dx - \int_{\frac{3}{4}}^1 0 dx = \frac{1}{4h^2} = 4$$

nach Berechnung der anderen Einträge erhalten wir als Matrix A:

$$A = \begin{pmatrix} A_{22} & A_{23} & A_{24} \\ A_{32} & A_{33} & A_{34} \\ A_{42} & A_{43} & A_{44} \end{pmatrix} = -\frac{1}{h^2} \begin{pmatrix} 1/2 & -\frac{1}{4} & 0 \\ -\frac{1}{4} & 1/2 & -\frac{1}{4} \\ 0 & -\frac{1}{4} & 1/2 \end{pmatrix} = \begin{pmatrix} -8 & 4 & 0 \\ 4 & -8 & 4 \\ 0 & 4 & -8 \end{pmatrix}$$

und für f:

$$f_2 = \int_0^1 f \cdot \phi_2 dx = -\frac{1}{h} \left(\int_0^{\frac{1}{4}} x dx + \int_{\frac{1}{4}}^{\frac{1}{2}} \left(\frac{1}{2} - x\right) dx \right) = -\frac{1}{h} \left(\frac{(\frac{1}{4})^2}{2} + \frac{1}{8} - \frac{(\frac{1}{2})^2}{2} + \frac{(\frac{1}{4})^2}{2} \right) = -\frac{1}{16h} = -\frac{1}{4}$$

f_3, f_4 analog, also erhalten wir:

$$f = \begin{pmatrix} f_2 \\ f_3 \\ f_4 \end{pmatrix} = \begin{pmatrix} -\frac{1}{h} \\ -\frac{1}{h} \\ -\frac{1}{h} \end{pmatrix} = \begin{pmatrix} -\frac{1}{4} \\ -\frac{1}{4} \\ -\frac{1}{4} \end{pmatrix}$$

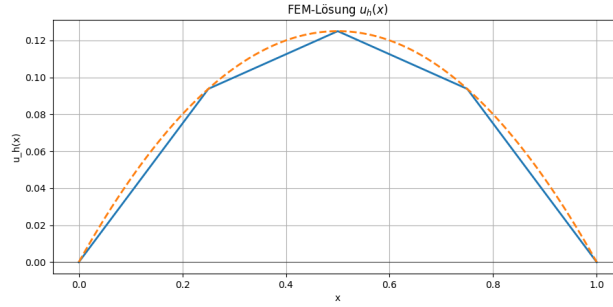
Nun müssen wir noch das erhaltene Gleichungssystem lösen:

$$\begin{pmatrix} -8 & 4 & 0 \\ 4 & -8 & 4 \\ 0 & 4 & -8 \end{pmatrix}^T \cdot \begin{pmatrix} u_2 \\ u_3 \\ u_4 \end{pmatrix} = \begin{pmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{pmatrix}$$

wir erhalten: $u = (0, \frac{3}{32}, \frac{1}{8}, \frac{3}{32}, 0)$ also ist unsere errechnete Lösung:

$$u_h(x) = \frac{3}{32} \cdot \phi_2(x) + \frac{1}{8} \cdot \phi_3(x) + \frac{3}{32} \cdot \phi_4(x)$$

Die errechnete Lösung, also unsere Näherungslösung, können wir nun mit der analytischen Lösung $u(x) = \frac{1}{2}x(1-x)$ vergleichen:



Es versteht sich, dass die Genauigkeit der Näherungslösung von der Anzahl unserer Stützstellen bzw. Basisfunktionen abhängt.

3.4 Randbedingungen in der FEM

Bzgl. der Randbedingungen für PDEs wird in der FEM je nach Art der Bedingung unterschiedlich umgegangen. Wir werden hier nur die drei grundlegenden Arten an Randbedingungen betrachten, nämlich Dirichlet-, Neumann- und Robin-Randbedingungen.

Dazu verwenden wir folgendes Beispiel, wobei $\partial\Omega := \Gamma_D \cup \Gamma_N \cup \Gamma_R$:

$$\begin{aligned} \Delta u &= f & \forall x \in \Omega \\ u &= u_D & \forall x \in \Gamma_D \\ \frac{\partial u}{\partial n}(x) &= g(x) & \forall x \in \Gamma_N \\ \alpha u(x) + \frac{\partial u}{\partial n}(x) &= \alpha u_0 & \forall x \in \Gamma_R \end{aligned}$$

Wir erhalten die schwache Form:

$$\int_{\Omega} \nabla u \nabla v \, dx - \int_{\Gamma_D \cup \Gamma_N \cup \Gamma_R} \frac{\partial u}{\partial n}(x) v \, ds = \int_{\Omega} f v \, dx \quad \forall v \in H^1(\Omega)$$

Die **Neumann- und Robinrandbedingungen** werden auch natürliche Randbedingungen genannt, da sie in der schwachen Formulierung der PDE enthalten sind und somit ganz 'natürlich' mit ihnen gearbeitet werden kann.

Lassen wir in unserem Beispiel die Dirichletrandbdg. kurz weg, können wir unser Problem wie folgt umschreiben:

$$\int_{\Omega} \nabla u \nabla v \, dx - \int_{\Gamma_N} g v \, ds + \int_{\Gamma_R} \alpha(u - u_0) v \, ds = \int_{\Omega} f v \, dx \quad \forall v \in H^1(\Omega)$$

da die Terme ' gv ' und ' $\alpha(u - u_0)v$ ' linear sind, können wir sie auf die rechte Seite bringen:

$$\int_{\Omega} \nabla u \nabla v \, dx + \int_{\Gamma_R} \alpha u v \, ds = \int_{\Gamma_N} g v \, ds + \int_{\Gamma_R} \alpha u_0 v \, ds + \int_{\Omega} f v \, dx \quad \forall v \in H^1(\Omega)$$

Wodurch wir wieder unsere unten angeführten, charakteristischen beiden Seiten erhalten, die wir, wie oben bereits erklärt und im Beispiel vorgerechnet, auf ein LGS zurückführen und somit lösen können.

$$\begin{aligned} A(u, v) &:= \int_{\Omega} \nabla u \nabla v \, dx + \int_{\Gamma_R} \alpha u v \, ds \\ f(v) &:= \int_{\Omega} f v \, dx + \int_{\Gamma_N} g v \, ds + \int_{\Gamma_R} \alpha u_0 v \, ds \end{aligned}$$

Die **Dirichletrandbedingungen** hingegen, werden, wie auch in der Theorie bereits in unserem Funktionenraum festgelegt, dh. die Skalare u_i für unsere gesuchte numerische Lösung u_h , bei denen sich die Basisfunktion b_i am Dirichletrand befindet, werden gleich null gesetzt. In der Theorie kennen wir den dazu analogen Raum $H_{\Gamma_D}^1(\Omega) := \{v \in H^1(\Omega) : v|_{\Gamma_D} = 0\}$, wobei die Funktioneneinschränkung als Spurenoperator zu verstehen ist.⁸

⁸[Jün01] S.12-14,[Scha]

4 Simulation mit FEniCS

FEniCS ist eine Open Source Plattform zum automatischen Lösen von PDEs. Dazu wird der Code, der sowohl in C++ als auch in Python geschrieben werden kann, in ein Finite-Elemente-Konzept übertragen und dieses dann automatisiert gelöst.

4.1 Softwareüberblick

In FEniCS müssen PDEs zuerst mithilfe der schwachen Formulierung an das Programm übergeben werden. Dazu ist es auch wichtig, in den richtigen Funktionenräumen zu arbeiten. Meistens werden dafür die im Bereich der PDEs sehr beliebten Sobolevräume verwendet, in unserem speziellen Fall benutzen wir Taylor-Hood-Räume.

Mit diesen Daten, also der schwachen Formulierung und dem dazugehörigen Funktionenraum erstellt FEniCS ein FEM-Konzept.⁹

Die Daten werden daraufhin auf einem sogenannten Mesh ausgewertet. Dazu wird der Raum in üblicher FEM-Manier unterteilt und auf jedem dieser einfacher zu behandelnden Elemente mit Lagrange Polynomen approximiert.

Die schwache Formulierung wird mithilfe der "unified form language (UFL)" an FEniCS übergeben. In dieser Programmiersprache können mathematisch relevante Operatoren, wie zum Beispiel Gradienten oder Skalarprodukt, direkt übergeben werden. Aus diesen werden dann Systemmatrizen effizient assembliert. Dadurch entsteht ein lineares oder nichtlineares Gleichungssystem, das im besten Fall auf jedem der Elemente lösbar ist. Auch Randbedingungen können in der Assemblierung berücksichtigt werden.

Zum Lösen der Systeme arbeitet FEniCS mit anderen Solverbibliotheken, wie zum Beispiel PETScs. Nichtlineare Probleme können beispielsweise mit Newton-Solver behandelt werden.

4.2 Setup und Codeaufbau

4.2.1 Aufbau

Zu Beginn wurde Deepseek gefragt, wie es das Projekt realisieren würde mit folgendem Anfangsbefehl:

Ich will ein Projekt erstellen, in dem ich die Navier-Stokes-Gleichungen mit FEniCS lösen und visualisieren lasse. Das Ziel ist einen Kanal mit einem Zylinder in der Mitte zu simulieren. Was muss ich dabei zuvor

⁹[LMW11]

beachten? Was für Files werden benötigt?

Dafür wurde mir folgender Aufbau empfohlen:

```
navier_stokes_project /
  venv/ oder .conda/
  requirements.txt
  README.md

src /
  geometry.py
  parameters.py
  solver.py
  boundary_conditions.py

meshes /
  cylinder_flow.xml

results /
  solutions /

tests /
  test_solver.py
```

Daraufhin habe ich die KI gebeten, jede dieser Dateien damit zu füllen, was sie für richtig hält. Dabei habe ich unterschiedliche KI-Modelle benutzt.

4.2.2 Funktionsweise

ChatGPT gab mir sehr schnell Inhalte für die einzelnen Dateien, die auf den ersten Blick sinnvoll wirkten. Bei genauerem Hinsehen konnte ich aber feststellen, dass einige Funktionen doppelt in verschiedenen Dateien definiert waren. Beispielsweise waren die Randbedingungen nicht nur in der vorgesehenen Datei "Boundary Conditions", sondern auch im allgemeinen Solver eingetragen. Dadurch entstanden schon beim ersten Generieren einige Konflikte. Diese ließen sich aber schnell beheben und eine funktionierende Grundstruktur war somit generiert. Im 'geometry'-File wurde der Kanal und die Art und Weise, in der das Mesh generiert werden sollte, definiert. Das Hindernis wird hier mithilfe einwa "Lochs" integriert. Im File 'parameters' wurden alle Größen definiert, die im Laufe der Simulation nicht verändert werden sollten. Das eigentliche Herzstück war die Datei 'solver'. Darin wurde das System der PDEs gelöst. Zu Beginn wird dabei mit der Stokes-Gleichung eine stationäre Lösung gesucht. Mit dieser wird anschließend mithilfe von Picard- und Zeititerationen die weitere Simulation durchgeführt. Die Daten werden daraufhin in eine Mesh-Datei übergeben, wodurch sich der Kanal visualisieren lässt.

4.3 Fazit

Im Verlauf der Arbeit wurden unterschiedliche KI-Modelle verwendet. Vor allem mit ChatGPT und Claude Sonnet wurden gute Erfahrungen gemacht. Beide Modelle erkannten Fehler gut. Die vorgeschlagenen Bugfixes haben leider nicht immer funktioniert, erst nach dem Hinweis, wo sich der Fehler befinden könnte, hat es meist funktioniert. Trotzdem hat der Code im Endeffekt funktioniert und eine erfolgreiche Visualisierung ausgegeben.

Das Hauptproblem bei der Simulation mit FEniCS war die Implementierung der Randbedingungen. Dabei wurden immer wieder Fehler eingebaut, wodurch die Simulation entweder nach einigen Zeititerationen divergierte oder laminar blieb. Auch die Berechnung der Reynoldszahl war anfangs falsch implementiert. Alles in allem ist trotzdem zu sagen, dass AI gut mit FEniCS arbeiten kann. Der Code ist leider ziemlich lang, einigermaßen unleserlich und nicht sehr effizient. Somit ist die Simulation mit AI eine gute Einführung in FEniCS und die Navier-Stokes-Gleichungen, allerdings würde ich nicht vorschlagen größere Projekte in diesem Stil durchzuführen.

5 Simulation mit NGSolve

NGSolve ist eine Open Source Software bzw. Bibliothek, welche mithilfe der Finite-Elemente-Methode PDEs löst. Sie basiert auf der Mesh-Generierungssoftware Netgen und wird aufgrund der mathematischen Genauigkeit vor allem für wissenschaftliche Rechnungen und Simulationen verwendet.¹⁰

5.1 Softwareaufbau

Im Allgemeinen wird in einer Simulation mit NGSolve zu Beginn die Geometrie festgelegt und darauf ein Mesh erstellt.

Danach werden die Funktionenräume definiert, da mit der schwachen Formulierung gearbeitet wird, ist das meist eine Form von Sobolevräumen. Dabei bestimmt das Programm automatisch die zu verwendenden Basisfunktionen, also ob lineare, quadratische oder höhergradig Funktionen beispielsweise.

Danach implementiert man noch die schwache Formulierung, die das Programm mithilfe von eingebauten Funktionen in Matrizen bzw. Vektoren umwandelt. Je nach Stabilität des Gleichungssystem kann sich der User anschließend selber für eine Lösungsmethode entscheiden. Die Lösung kann anschließend mithilfe von einfachen Befehlen visualisiert werden.

5.2 Simulation mit künstlicher Intelligenz

Der folgende Anfangsbefehl wurde an ChatGPT 5.2, Deepseek und Claude Sonnet 4.5 geschickt:

*Hallo, ich habe eine Aufgabe für dich.
Kannst du mir bitte einen Code für ein Jupyter-Notebook erstellen, der mithilfe von NGSolve eine Strömung simuliert. Dazu soll er die Navier-Stokes-Gleichungen mithilfe der Finite-Elemente-Methode lösen.
Die Strömung soll dabei von einem zylinderförmigen Hindernis gestört werden, was die Simulation berücksichtigen und somit auch die erzeugten Strudel nach dem Hindernis abbilden soll. (Karmansche Wirbelstraße)*

Da Deepseek einen sehr viel komplizierteren Code als die anderen beiden KI's zurückgab, der wie die Codes der beiden anderen nicht funktionierte, wurde die Arbeit damit sofort eingestellt.

Aus eigener Gewohnheit entschied ich mich mit ChatGPT weiterzuarbeiten. Es gab kleine, schnell behobene Probleme mit Geometrie und Mesh, die nur aufgrund von der Verwechslung der verschiedenen Versionen von NGSolve aufkamen.

Außerdem war das Lösen der Stokes-Gleichungen, die als unsere Anfangslösung verwendet wurde, lange ein Problem, konnte allerdings auch nach einiger Zeit gelöst werden.

¹⁰[Schb]: NGSolve

Das größte Problem stellte das Lösen der Navier-Stokes-Gleichungen an sich dar. Die Berechnung war zwar von Beginn an sinnvoll, funktionierte allerdings, aufgrund von zu falschen Zeitpunkten gesetzte Randbedingungen lange nicht. Das konnte ChatGPT nicht lösen, sondern war die Arbeit von Claude Sonnet 4.5 (Copilot) war.

5.3 Fazit

Obwohl die Simulation in NGSolve mit KI erfolgreich war, braucht es nach wie vor jemanden, der einen groben Überblick über die Finite-Elemente-Methode, PDEs und dem Umgang mit dazugehörigen Randbedingungen, Numerik und generell den betroffenen mathematischen Bereiche hat, um die Fehler der KI zu identifizieren.

Der Code der KI ist sehr gut strukturiert und übersichtlich, außerdem besteht die gesamte Simulation aus 97 Codezeilen.

6 Finite-Volumen-Methode (FVM)

6.1 Was ist die FVM?

Die Finite-Volumen-Methode ist ein numerisches Verfahren zur Lösung partieller Differentialgleichungen, denen ein Erhaltungssatz zugrunde liegt.

Die Grundidee des Verfahrens besteht darin, das Gebiet in endlich viele Kontrollvolumina (Finite Volumen) zu unterteilen und über diese zu integrieren.

Im Gegensatz zur FEM, welche mit der schwachen Formulierung arbeitet, basiert die FVM auf einem konstruktiven Ansatz. Je nachdem welcher Term der Gleichung betrachtet wird, verwendet man verschiedene Diskretisierungsverfahren, um ein großes Gleichungssystem zu erhalten.

6.2 Herleitung anhand eines einfachen Beispiels

Wir betrachten folgende allgemeine Art einer Erhaltungsgleichung:

$$\frac{\partial u}{\partial t}(x, t) + \nabla \cdot f(u(x, t)) = 0 \quad \text{auf } \Omega \subset \mathbb{R}^d \times [0, \infty)$$

Zuerst wird das Gebiet in Kontrollvolumina unterteilt, wobei in jeder dieser Zellen die Gleichung gilt.

Danach wird über die einzelnen Zellen integriert:

$$\int_{\Omega_i} \frac{\partial u}{\partial t} d\Omega + \int_{\Omega_i} \nabla \cdot f(u) d\Omega = 0$$

Für den zweiten Term auf der linken Seite bietet sich der Gaußsche Integralsatz an. Wir erhalten für diesen Term:

$$\int_{\partial\Omega_i} f(u) \cdot \mathbf{n} dS$$

wobei $\mathbf{n}$ die äußere Einheitsnormale ist.

Der Mittelwert in jeder Zelle lässt sich schreiben als:

$$u_i = \frac{1}{|\Omega_i|} \int_{\Omega_i} u d\Omega$$

und da sich die Zellen mit der Zeit nicht verändern:

$$\frac{\partial u_i}{\partial t} = -\frac{1}{|\Omega_i|} \int_{\partial\Omega_i} f(u) \cdot \mathbf{n} dS$$

Dies liefert eine Grundlage für die FVM.¹¹

In den nächsten Schritten wird das Flächenintegral diskretisiert. Im Allgemeinen sind die Kontrollvolumina polyedrisch. Die Randintegrale lassen sich daher als Summe über die Zellflächen schreiben:

$$\sum_{i=1}^N \int_{S_i} f(u) \cdot \mathbf{n}_i dS$$

Die Hauptannahme der FVM besagt, dass die Änderung innerhalb der Zelle und an den Zellwänden nur linear ist, so lässt sich das Integral umschreiben auf den Mittelwert mal der Fläche.

Folgendes Kapitel befasst sich genauer mit den Diskretisierungen im Fall der inkompressiblen Navier-Stokes-Gleichungen.

6.3 Diskretisierungen im speziellen Fall

Die nachstehende Erhaltungsgleichung dient als Grundlage:

$$\frac{\partial u_k}{\partial t} + \nabla \cdot (\mathbf{u} u_k) = \nabla \cdot (\mu \nabla u_k) + Q$$

Hier ist u_k die k-te Komponente des Geschwindigkeitsvektors $\mathbf{u}$ und Q der Quellterm, welcher unter anderem $-\nabla p$ enthält.

Im Weiteren wird $\phi := u_k$ zur besseren Lesbarkeit gesetzt.

6.3.1 Der Quellterm

Der Quellterm wird in den Diskretisierungen für $\mathbf{u}$ als konstant angesehen:

$$\int_{V_P} Q dV \approx Q_P V_P$$

6.3.2 Der lokale Zeitterm

Da sich die Kontrollvolumina mit der Zeit nicht verändern, lässt sich das Integral mit der Ableitung vertauschen. Mit der Annahme, dass die Änderung innerhalb der Zelle nur linear ist, wird das Integral zu Mittelwert mal Volumen:

$$\int_{V_P} \frac{\partial \phi}{\partial t} dV \approx \frac{\partial}{\partial t} (\phi_P V_P) = V_P \frac{\partial \phi_P}{\partial t}$$

¹¹[Jün26]

Um die Zeit zu diskretisieren, gibt es verschiedene Methoden. Im Folgenden arbeiten wir mit dem impliziten Euler.

$$\left. \frac{\partial \phi}{\partial t} \right|^{n+1} \approx \frac{\phi_P^{n+1} - \phi_P^n}{\Delta t}$$

Somit wird der Zellbeitrag des Zeitterms zu:

$$V_P \frac{\phi_P^{n+1} - \phi_P^n}{\Delta t}$$

6.3.3 Der Konvektionsterm

Nach Integration über die Kontrollvolumina bietet sich der Gaußsche Integralsatz an.

$$\int_{V_P} \nabla \cdot (\mathbf{u}\phi) dV = \int_{\partial V_P} (\mathbf{u}\phi) \cdot \mathbf{n} dS$$

Der Rand ∂V_P wird durch die Zellflächen $f \in \mathcal{F}_P$ partitioniert. Wir summieren über diese Flächen und verwenden erneut nur lineare Änderung:

$$= \sum_{f \in \mathcal{F}_P} \int_{S_f} (\mathbf{u}\phi) \cdot \mathbf{n}_f dS \approx \sum_{f \in \mathcal{F}_P} (\mathbf{u}_f \cdot \mathbf{n}_f) \phi_f |S_f|$$

wobei hier ϕ_f und $\mathbf{u}_f$ für den Mittelwert einer bestimmten Zellfläche stehen. In dieser Form ist der Konvektionsterm nichtlinear, da die Geschwindigkeit sowohl als Transportgeschwindigkeit als auch als transportierte Größe ϕ auftritt.

Wir definieren $F_f := (\mathbf{u}_f \cdot \mathbf{n}_f) |S_f|$ als den (Volumen-)Durchfluss durch die Fläche f und verwenden für diesen, das aus dem vorherigen Schritt bekannte Geschwindigkeitsfeld, um für jeden Teilschritt ein lineares Gleichungssystem zu erhalten.

Somit wird der Konvektionsterm zu:

$$\sum_{f \in \mathcal{F}_P} F_f \phi_f$$

6.3.4 Der Diffusionsterm

Wie beim Konvektionsterm wird mithilfe des Gaußschen Integralsatzes umgeformt. Anschließend werden die Flächenintegrale, erneut unter der Annahme linearer Variation, als Mittelwert mal Fläche approximiert:

$$\int_{V_P} \nabla \cdot (\mu \nabla \phi) dV = \int_{\partial V_P} \mu \nabla \phi \cdot \mathbf{n} dS = \sum_{f \in \mathcal{F}_P} \int_{S_f} \mu \nabla \phi \cdot \mathbf{n}_f dS \approx \sum_{f \in \mathcal{F}_P} \mu_f \nabla \phi_f \cdot \mathbf{n}_f |S_f|$$

μ_f bezeichnet die auf der Fläche ausgewertete Viskosität, im hier betrachteten Fall ist μ konstant, sodass $\mu_f = \mu$ gilt.¹²

Wir bemerken, dass der Diffusionsterm, wie auch der Konvektionsterm, Mittelwerte der Zellflächen beinhaltet. Jedoch werden die unbekannten Größen jeweils nur an den Zellmittelpunkten gespeichert.

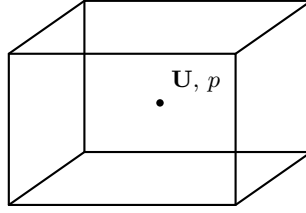


Abbildung 3: Zellzentrierte Speicherung der Unbekannten.

6.3.5 Unterschiedliche Interpolationsschemata

Die FVM verwendet unterschiedliche Interpolationsschemata, um Werte an den Zellflächen zu erhalten, unter anderem Linear, Upwind, sowie Kombinationen der beiden. Als Beispiel für den Konvektionsterm betrachten wir das Upwind-Schema. Als Ausgangspunkt dient:

$$\sum_{f \in \mathcal{F}_P} F_f \phi_f$$

Das Schema basiert auf folgender Idee: Je nach Richtung des Flusses durch die Zellfläche, wird der Zellmittelwert der stromaufwärtigen Zelle als Wert auf der Fläche angesetzt.

$$\phi_f = \begin{cases} \phi_P, & F_f > 0, \\ \phi_N, & F_f < 0. \end{cases}$$

¹²[VM07] S.24-26, [Jün26]

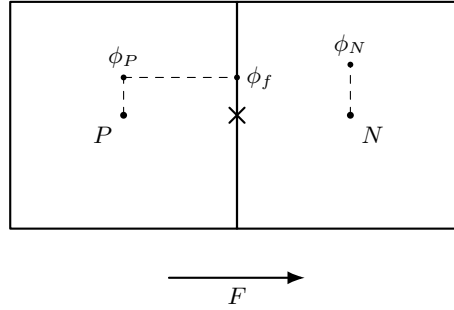


Abbildung 4: Skizze zweier benachbarter Kontrollvolumina und der zugehörigen Größen.

Upwind ist ein Verfahren 1. Ordnung, das bedeutet, es ist nicht besonders präzise, dafür aber sehr stabil.¹³

Für den Diffusionsterm verwenden wir das lineare Schema (Central Differencing). Ausgehend von

$$\sum_{f \in \mathcal{F}_P} \mu_f \nabla \phi_f \cdot \mathbf{n}_f |S_f|$$

lässt sich der Normalgradient auf der Fläche f durch

$$\nabla \phi_f \cdot \mathbf{n}_f \approx \frac{\phi_N - \phi_P}{d_{PN}}$$

approximieren, wobei $d_{PN} = \|\mathbf{x}_N - \mathbf{x}_P\|$. Somit wird der Term zu:

$$\sum_{f \in \mathcal{F}_P} \mu_f |S_f| \frac{\phi_N - \phi_P}{d_{PN}}$$

Diese Approximation funktioniert gut, solange die Gerade $\mathbf{x}_P \rightarrow \mathbf{x}_N$ auf dem Normalvektor liegt.

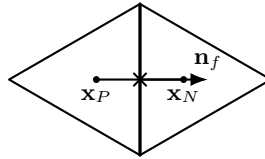


Abbildung 5: Orthogonale Anordnung: Die Verbindung der Zellzentren ist parallel zur Flächennormalen.

¹³[Wol20]

Im Allgemeinen sind benachbarte Zellen nicht notwendigerweise orthogonal zueinander angeordnet. Um die daraus entstehenden Diskretisierungsfehler zu reduzieren, werden in vielen Solver-Implementierungen Non-orthogonal correctors eingesetzt. Auf die Details dieser Korrekturverfahren wird im Rahmen dieser Arbeit nicht weiter eingegangen.

6.4 Gute Mesh-Konstruktion

Trotz dieser Korrekturmöglichkeiten besteht die grundlegende Strategie darin, ein möglichst qualitativ hochwertiges Mesh zu erzeugen. Wir achten besonders auf Orthogonalität, langsames Zellwachstum und geringe Verzerrung.¹⁴

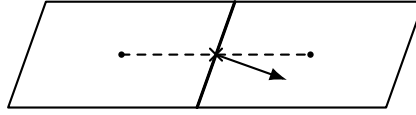


Abbildung 6: Non-orthogonality: Die Verbindung der Zellzentren ist nicht parallel zur Flächennormalen.

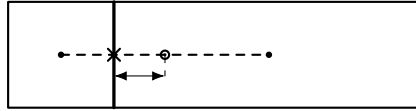


Abbildung 7: Fast volume growth: Face-Mittelpunkt und Streckenmittelpunkt liegen weit auseinander.

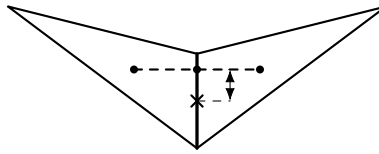


Abbildung 8: Skewness: Schnittpunkt liegt nicht im Face-Mittelpunkt.

6.5 Verfahren der Solver

Nach der Diskretisierung entstehen algebraische Gleichungen, die pro Zeitschritt bzw. Iteration gelöst werden. Da der Konvektionsterm nichtlinear ist und Geschwindigkeit sowie Druck gekoppelt sind, erfolgt die Lösung typischerweise in einem iterativen, getrennten Ablauf.

Aus einem aktuellen Geschwindigkeitsfeld $\mathbf{u}$ wird zunächst der Fluss über die Zellflächen ϕ bestimmt. Mit diesem vorläufig gegebenen ϕ wird anschließend

¹⁴[MMD16] S.239-254

die Impulsgleichung gelöst, wodurch ein Zwischenfeld $\mathbf{u}^*$ entsteht. Darauf aufbauend wird eine Druckkorrekturformulierung hergeleitet, die die Kontinuitätsbedingung sicherstellt, und die Felder p , $\mathbf{u}$ und ϕ werden korrigiert. Dieser Korrekturschritt wird wiederholt, bis die Residuen ausreichend klein sind.

6.5.1 Druckkorrektur

Da Geschwindigkeit und Druck über die Inkompressibilitätsbedingung $\nabla \cdot \mathbf{u} = 0$ gekoppelt sind, wird häufig ein separater Korrekturansatz verwendet. Zunächst wird aus einem vorläufigen Druckfeld $p^{(m)}$ (Druck aus der m -ten Iteration) eine Impulsgleichung gelöst, was ein Zwischenfeld $\mathbf{u}^*$ liefert. Dieses Zwischenfeld erfüllt die Kontinuitätsbedingung im Allgemeinen noch nicht, d.h. $\nabla \cdot \mathbf{u}^* \neq 0$.

Daher wird eine Druckkorrektur $p' = p - p^{(m)}$ eingeführt und so bestimmt, dass die korrigierten Größen $\mathbf{u} = \mathbf{u}^* - \alpha \nabla p'$ divergenzfrei werden. Das führt auf eine Poisson-ähnliche Gleichung für p' , die aus der Forderung $\nabla \cdot \mathbf{u} = 0$ entsteht. Nach dem Lösen dieser Korrekturgleichung werden $p, \mathbf{u}$ und die Flüsse ϕ aktualisiert, der Vorgang kann iteriert werden, bis der Restfehler der Kontinuität hinreichend klein ist.¹⁵

¹⁵[Aps25]

7 Simulation mit OpenFOAM

OpenFOAM ist eine Open-Source Software für Computational Fluid Dynamics. Sie basiert auf C++ Libraries und hat ein breites Anwendungsgebiet. In OpenFOAM sind verschiedenste Solver für unterschiedliche Problemstellungen implementiert. Die Software basiert hauptsächlich auf der Finite-Volumen-Methode. FEM kann bei Bedarf über die Kopplung mit externen FEM-Programmen eingebunden werden. Zusätzlich gibt es mit FiniteArea eine Methode für Berechnungen auf Oberflächen, die der FVM konzeptionell ähnlich ist.

7.1 Überblick und Codeaufbau

Die Software operiert auf einem Case-Ordner, in diesem werden vorab alle Parameter, Diskretisierungsmethoden, sowie die Geometrie und die gewünschten Solver festgelegt. Die Anfangs- und Randbedingungen sind im Ordner 0/ definiert. Bei der Ausführung wird für jeden Zeitschritt ein Ordner erstellt, in welchem die Werte der Felder aufzufinden sind.

7.1.1 Aufbau

Zu Projektstart wurde ChatGPT gefragt, wie es die Umsetzung des Projekts vorschlagen würde. Der initiale Prompt lautete:

Hallo, ich möchte eine Wasserströmung mit den Navier-Stokes-Gleichungen simulieren und dafür mit OpenFOAM arbeiten. Benutzen möchte ich gerne die Finite-Volumen-Methode. Kannst du mir bitte eine Anleitung für den Grundaufbau für einen solchen Simulations-Code mit OpenFOAM geben?

Daraufhin wurde folgender Projektaufbau vorgeschlagen:

```
NavierStokes.OpenFOAM/  
  0/  
    p  
    U  
  constant/  
    transportProperties  
    turbulenceProperties  
    triSurface/  
      cylinder.stl  
  system/  
    blockMeshDict  
    controlDict  
    fvSchemes  
    fvSolution  
    snappyHexMeshDict  
Allclean  
Allrun
```

Nachdem die Ordnerstruktur erstellt war, ließ ich ChatGPT die Dateien jeweils einzeln schreiben.

7.1.2 Funktionsweise

Der OpenFOAM-Case folgt der üblichen Verzeichnisstruktur. Die Anfangs- und Randbedingungen der Felder `U` und `p` sind im Ordner `0/` abgelegt. Zeitunabhängige Eigenschaften wie Materialparameter (`transportProperties`) und Turbulenzeinstellungen (`turbulenceProperties`) befinden sich in `constant/`, zusätzlich liegt die für `snappyHexMesh` verwendete Geometrie als STL-Datei in `constant/triSurface/`. Die numerischen Einstellungen und die Ablaufsteuerung sind in `system/` definiert, insbesondere das Grundgitter (`blockMeshDict`), die Snapping- und Verfeinerungsparameter (`snappyHexMeshDict`) sowie Zeitschritt- und Solverkonfigurationen (`controlDict`, `fvSchemes`, `fvSolution`). Die Skripte `Allrun` und `Allclean` dienen schließlich zum automatisierten Ausführen bzw. Bereinigen des Cases.

ChatGPT wollte zuerst mit einem 2D-Mesh arbeiten, jedoch lässt sich in diesem Fall der Zylinder nicht mehr verfeinert, mit `snappyHexMesh`, aus dem Gitter schneiden. Es schlug daraufhin vor, mit einem dünnen 3D-Mesh zu arbeiten, was gut funktionierte.

7.2 Fazit

Die Struktur des Codes ist gut erkennbar und die Anzahl der Code-Zeilen hält sich in Grenzen, was das Arbeiten mit KI grundsätzlich erleichtert. ChatGPT 5.2 konnte von Anfang an sehr gut mit OpenFOAM arbeiten. Kleine Fehler wurden rasch erkannt, und ihre Behebung nahm nur wenig Zeit in Anspruch. Deswegen sah ich keine Notwendigkeit auf andere KI-Modelle zurückzugreifen.

ChatGPT gab auch Vorschläge für Hilfestellungen, wie zum Beispiel die OpenFOAM-Tutorials und das erstellen der `Allrun/Allclean` files. Nur am Anfang wurde ein veralteter Solver vorgeschlagen, aber auch dieses und Folgeprobleme ließen sich relativ schnell beheben.

Gesamt empfand ich das Arbeiten mit KI unter Verwendung von OpenFOAM als angenehm und würde auch weitere Projekte dieser Art durchführen.

Referenzen

- [Jün01] Prof. Ansgar Jüngel. *Das kleine Finite-Elemente-Skript*. Skript, PDF. 2001. URL: <https://www.tuwien.at/index.php?eID=dumpFile&t=f&f=188856&token=858e472985ae8b68f353c303310259da177d0de6>.
- [VM07] H. K. Versteeg and W. Malalasekera. *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*. 2nd ed. Pearson Education, 2007.
- [LMW11] Andreas Logg, Kent Andre Mardal, and Garth N. Wells. *Automated Solution of Differential Equations by the Finite Element Method*. PDF. 2011.
- [MMD16] F. Moukalled, L. Mangani, and M. Darwish. *The Finite Volume Method in Computational Fluid Dynamics*. Springer, 2016.
- [Rv19] R. Verfürth. *Finite Element Methoden für Navier-Stokes Gleichungen*. Skript, PDF. 2019.
- [Jün20] Prof. Ansgar Jüngel. *Nichtlineare partielle Differentialgleichungen*. Skript, PDF. 2020.
- [Wol20] WolfDynamics. *A Crash Introduction to the Finite Volume Method*. Workshop Notes, PDF. 2020. URL: https://www.wolfdynamics.com/training/OF_WS2020/traning_session2020.pdf.
- [Alf24] Cristian Marchiolo Alfredo Soldati. *Fluid Mechanics for Mechanical Engineers*. Skript, PDF. 2024.
- [Aps25] David D. Apsley. *Pressure and Velocity (SIMPLE)*. Lecture Notes, PDF. 2025. URL: <https://personalpages.manchester.ac.uk/staff/david.d.apsley/lectures/comphydr/p5.pdf>.
- [Inn+25] Prof. M. Innerberger et al. *Numerics of differential equations*. Skript, PDF. Vorlesungsskript, zur Verfügung gestellt von einer Studienkollegin. 2025.
- [Jün26] Prof. Ansgar Jüngel. *Finite-volume methods and structure-preservation for cross-diffusion systems*. Skript, PDF. 2026. URL: <https://www.tuwien.at/index.php?eID=dumpFile&t=f&f=1452119&token=7fe8dd189d32afd385d039491eb990e185ce3e0e>.
- [Scha] Prof. Joachim Schöberl. *Boundary Conditions*. URL: https://jschoeberl.github.io/IntroSC/PDEs/Poisson/boundary_conditions.html. (accessed: 19.02.2026).
- [Schb] Prof. Joachim Schöberl. *ngsolve*. URL: <https://github.com/NGSolve/ngsolve>. (accessed: 19.02.2026).
- [Schc] Prof. Joachim Schöberl. *Solving the Poisson Equation*. URL: <https://github.com/NGSolve/ngsolve>. (accessed: 19.02.2026).
- [Sto] Malte Stonis. *Finite Elemente Methode kompakt erklärt*. URL: <https://www.iph-hannover.de/de/dienstleistungen/fertigungsverfahren/fem-simulation-cad-prozesskettenbetrachtung/fem/>. (accessed: 08.02.2026).

Eidesstattliche Erklärung

Wir erklären eidesstattlich, dass wir die vorliegende Seminararbeit selbstständig und ohne unerlaubte fremde Hilfe verfasst haben. Alle verwendeten Quellen und Hilfsmittel wurden vollständig angegeben, und wörtlich oder sinngemäß übernommene Stellen als solche gekennzeichnet. Die Arbeit wurde bisher weder im In- noch im Ausland in gleicher oder ähnlicher Form einer Prüfungsbehörde vorgelegt.

Da das Thema der Arbeit, die 'Simulation der Navier-Stokes-Gleichungen mithilfe von künstlicher Intelligenz' war, war der Einsatz von KI zur Entwicklung des Programmcodes, ausdrücklicher Bestandteil der Aufgabenstellung und entsprechend gestattet.

Anna Sonnleitner


Unterschrift

Emanuel Steininger


Unterschrift

Martin Strobl


Unterschrift